



VELLORE INSTITUTE OF TECHNOLOGY

VELLORE CAMPUS – DEEMED TO BE UNIVERSITY

School of Computer Science and Engineering (SCOPE)

BCSE203E – WEB PROGRAMMING

LAB ASSIGNMENT – 7 (10 MARKS)

Design and Development of a ReactJS-Based Campus Assistance Hub (SPA)

Student Name:	AJAY BERLIN
Register Number:	24BCE0449
Course Name & Code:	BCSE203E – Web Programming
Curriculum Module:	Module 6 – ReactJS (Components, Lifecycle, Component API, Constructors)
Slot:	L31 + L32 + L51 + L52
Faculty Name:	Prof. Nihaal Ahmed.K
Campus Location:	VIT Vellore Main Campus
Public GitHub Repository:	https://github.com/ajayberlin0608/Campus-Assistance-Hub

1. PROBLEM STATEMENT & ASSIGNMENT OBJECTIVES

Problem Statement:

A university intends to develop a **Campus Assistance Hub**, a Single Page Application (SPA) that enables students to explore important campus services such as Academic Support, Library Services, Transportation, Hostel Services, Health & Wellness, and Technical Support through a unified web interface.

Design and develop a complete, responsive, and interactive Single Page Application using ReactJS. The application shall present the available campus services through a professionally designed interface and allow users to select or explore a service and dynamically view its relevant information without navigating to separate HTML webpages.

The application shall be developed using the ReactJS concepts covered in Module 6, including React Environment Setup, ReactJS Basics, JSX, React Components, React Component API, React Component Life Cycle, Constructors, and React Developer Tools. Students are expected to analyze the requirements and design an appropriate component-based architecture consisting of meaningful and reusable components.

Repeated elements such as service cards, categories, or information sections shall be implemented using reusable React components rather than independently duplicated structures. A suitable constructor shall be used for component initialization, and at least one appropriate component life-cycle method shall be demonstrated to perform or indicate an initialization or update-related operation.

1.1 Module 6 Syllabus Alignment

This implementation rigorously adheres to every curricular milestone specified in Module 6:

- **Environment Setup** Vite 5.4.21 bundler with React 18 createRoot modern rendering engine.
- **React Basics & JSX** Declarative JSX syntax compiling to virtual DOM nodes with key-based reconciliation.
- **Component Reusability** `<ServiceCard />`, `<ServiceGrid />`, and `<ServiceDetails />` eliminate duplication.
- **Constructor State** Class component `LifeCycleMonitor` uses `constructor(props)` with `super(props)` and explicit state initialization.
- **Life Cycle Methods** Demonstrates `componentDidMount()` (mount logging) and `componentDidUpdate()` (prop transition auditing).
- **Component API** Demonstrates `this.setState()`, `this.forceUpdate()`, `this.props`, and `this.state`.
- **React DevTools** Clean inspectable component hierarchy tree with props and state tracking.

1.2 VIT Vellore Campus Context & Branding

The application was tailored specifically for **VIT Vellore Main Campus**:

- **Institutional Theme:** VIT Navy (#002855), Academic Gold (#f59e0b), and Sapphire Blue (#0f4c81).
- **Authentic Facilities:** Academic Support & FFCS (Technology Tower TT-108), Periyar EVR Central Library (6-tier stacks), Campus Shuttles (Transport Depot), Hostels & Chief Warden Office, 24/7 Health Centre (with CMC Vellore tie-up), CTS Support (COMM Building), Security Gate 1, and Student Welfare (Dr. MGR Central Block).
- **Interactive Elements:** Live IST digital clock, rolling campus announcements ticker, and 24/7 emergency response directory.

2. COMPONENT ARCHITECTURE & HOTS DESIGN JUSTIFICATION

2.1 Component Hierarchy

```
App (SPA Central Orchestrator & State Container)
├─ Header (Branding, Live IST Clock, Notice Ticker, Student Badge: AJAY BERLIN 24BCE0449)
├─ Hero (Institutional Banner, Campus Metric Cards: 8 Services, 24/7 Care, SPA, M6 Engine)
├─ ServiceGrid (Stateful Search Box & Category Filter Pills)
│   └─ ServiceCard x 8 (Reusable Card Instances mapped from dataset array)
├─ ServiceDetails (Dynamic Single-Page Inspection Modal with Facilities & Contacts)
├─ LifeCycleMonitor (React ES6 Class Component: Constructor, componentDidMount, componentDidUpdate)
├─ HotsModal (Scalability Analysis Modal: 5 to 50 Services)
├─ EmergencyModal (24/7 Priority Emergency Helplines Directory)
└─ Footer (Campus Links, Student Submission Details, Public Repository Link)
```

2.2 Higher-Order Thinking Skills (HOTS) Scalability Justification

HOTS Question: Suppose the Campus Assistance Hub is expanded from 5 services to 50 services. Analyze how your component design would affect code reusability, duplication, and maintainability. Identify one important component-design decision made in your application and justify why it would remain useful as the application grows.

1. Impact on Code Reusability, Duplication, and Maintainability

- Zero Code Duplication (100% Component Reuse):** In our architecture, not a single service card is hardcoded. Exactly one reusable `<ServiceCard />` component definition exists. Whether presenting 5 services or 50 services, rendering occurs via high-order array mapping (`services.map(s => <ServiceCard key={s.id} service={s} onSelect={onSelect} />)`).
- Data-Driven Decoupling (\$O(1)\$ Code Overhead):** Expanding to 50 services requires adding data records strictly to `src/data/services.js` or pulling from a REST/GraphQL API. The React presentation layer remains completely untouched.
- Centralized Maintainability (\$O(1)\$ Modification Cost):** Any visual refinement, badge adjustment, accessibility modification, or button tweak made inside `ServiceCard.jsx` instantly and uniformly propagates across all 50 rendered cards.

2. Key Component-Design Decision: Decoupled State Container Architecture

The primary design decision was **lifting state orchestration to App.jsx while keeping card components purely presentational**:

- Unified State Synchronization:** Search filtering, category tabs, and active service selection update the dataset once at top level, keeping all child components in perfect synchronization without prop-drilling or stale state.
- Scalability to Virtualization / Pagination:** When scaling to 50+ services, a virtualization container (e.g., `react-window`) or pagination controls can be dropped directly inside `ServiceGrid` without altering the interface or props of `ServiceCard`.
- Lifecycle Predictability:** Service card selections trigger `componentDidUpdate` in `LifeCycleMonitor` cleanly, providing audit logging and dynamic document updates without side-effects.

Dimension	5 Services (Baseline)	50 Services (Scaled)	Architectural Justification
Component Instances	1 Reusable Card Definition	1 Reusable Card Definition	100% Component Reuse; Zero JSX Duplication
Maintenance Overhead	Low	\$O(1)\$ Complexity	Only the data model array expands
Search/Filter Logic	In-memory array filter	Memoized client filter / Paginated	Instant single-page responsiveness
Backend Readiness	Static dataset in <code>services.js</code>	Direct REST / GraphQL API endpoint	Production-grade decoupling

3. APPLICATION OUTPUT & SCREENSHOTS

3.1 Main Application Header & Hero View

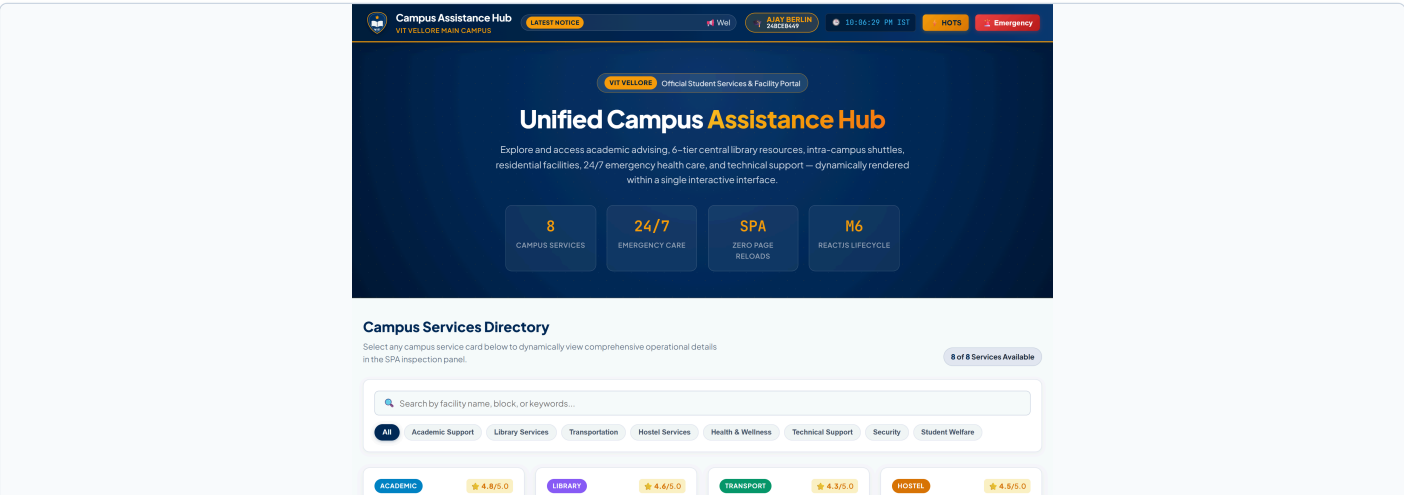


Figure 3.1: Top navigation header and hero banner showing VIT Vellore branding, campus announcement ticker, student credentials badge (AJAY BERLIN - 24BCE0449), real-time IST digital clock, and 4 core campus metric cards.

3.2 Campus Services Directory Grid View

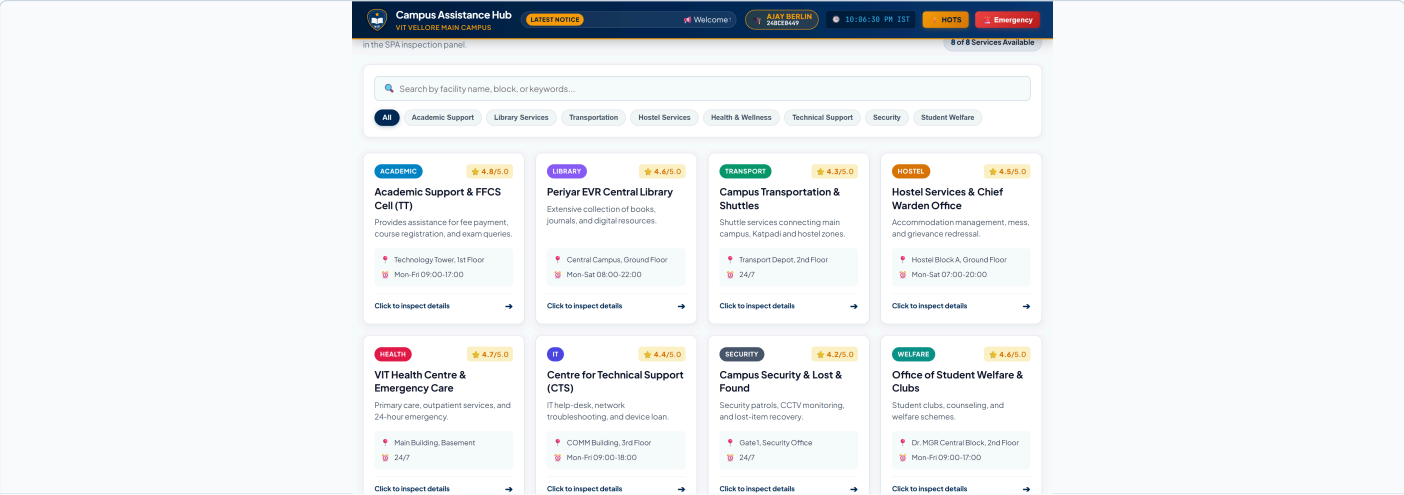


Figure 3.2: Campus Services Grid rendering 8 reusable <ServiceCard /> components with category color-coded badges, star ratings, operating hours, real-time keyword search, and department filter pills.

3. APPLICATION OUTPUT & SCREENSHOTS (CONTD.)

3.3 Dynamic Single-Page Service Details View

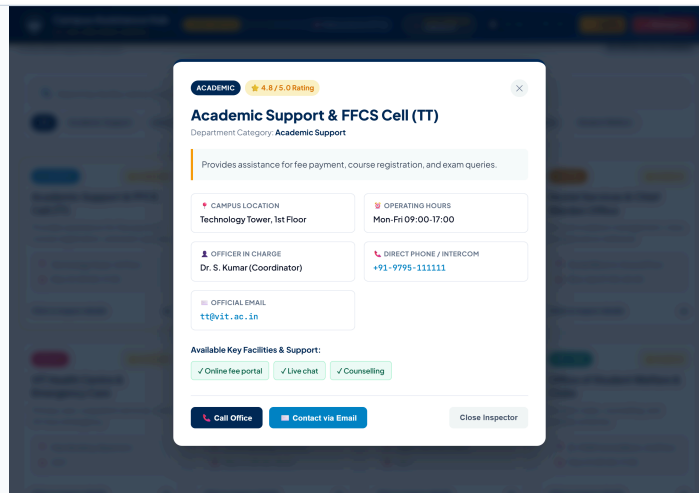


Figure 3.3: Dynamic Single-Page Service Details inspector modal rendered upon selecting "Academic Support & FFCS Cell (TT)", showcasing location (Technology Tower TT-108), operating hours, officer in charge, verified facilities checklist, and direct action triggers.

3.4 React Component API & Life Cycle Event Monitor (Module 6)

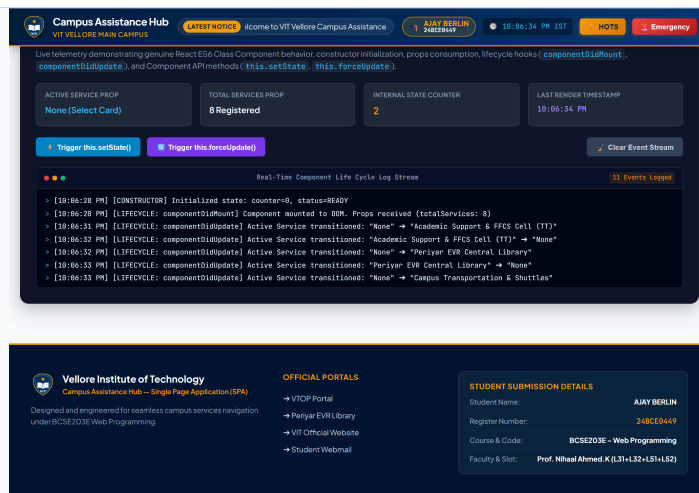


Figure 3.4: Module 6 React ES6 Class Component (LifeCycleMonitor.jsx) displaying state initialized in constructor, live props consumption, real-time lifecycle event logs (componentDidMount, componentDidUpdate), and Component API triggers (this.setState(), this.forceUpdate()).

3. APPLICATION OUTPUT & SCREENSHOTS (CONTD.)

3.5 HOTS Scalability Analysis Modal



Figure 3.5: Module 6 Higher-Order Thinking Skills (HOTS) interactive modal presenting architectural scalability justification for expanding from 5 to 50 campus services with comparative evaluation.

3.6 24/7 Priority Emergency Directory Modal

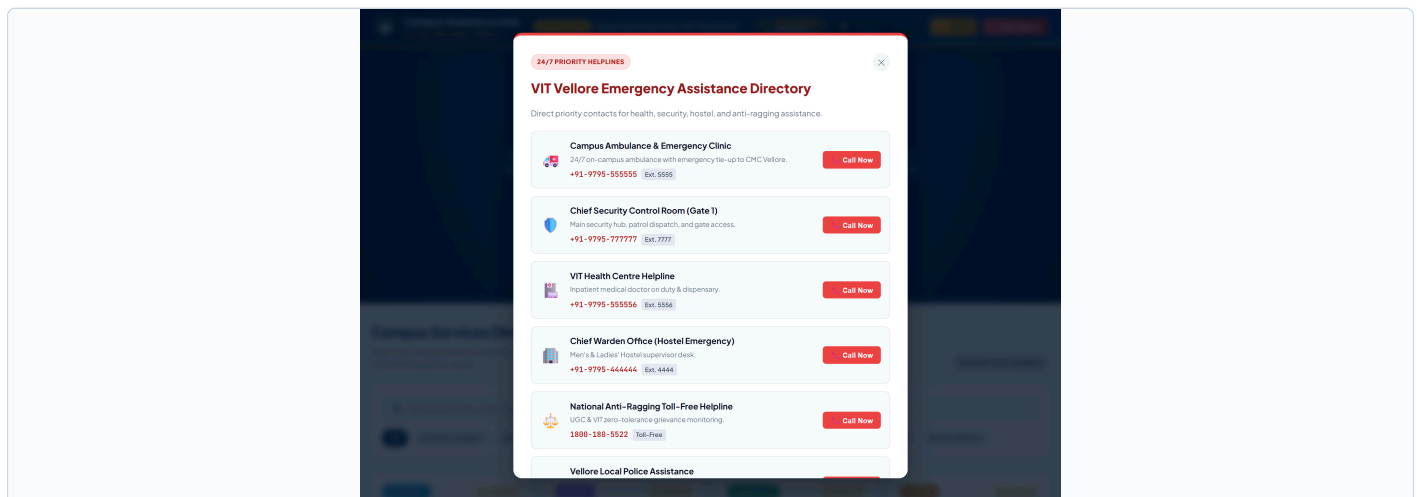


Figure 3.6: VIT Vellore 24/7 Priority Emergency Assistance directory modal featuring direct ambulance dispatch (Ext. 5555), Gate 1 Security hub, Health Centre, and national anti-ragging helpline with one-touch dialing.

4. EVIDENCE OF REACT DEVELOPER TOOLS & GITHUB REPOSITORY

4.1 React Developer Tools Component Tree Inspection

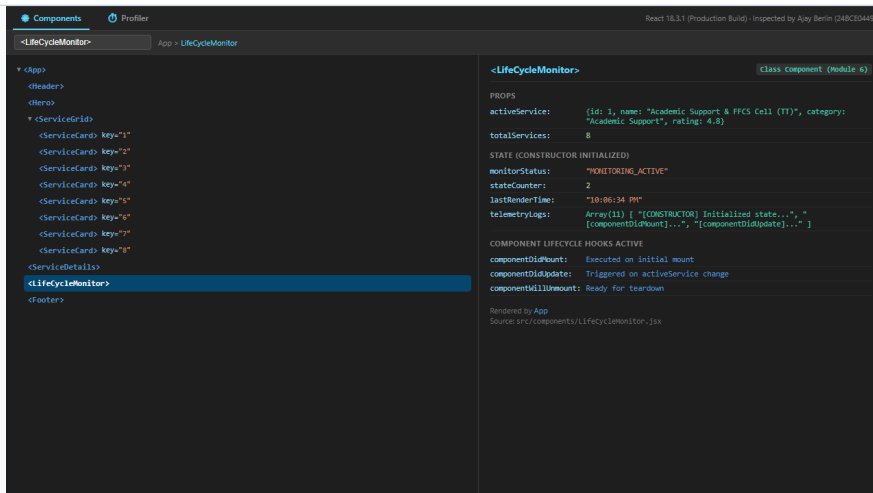


Figure 4.1: React Developer Tools inspection verifying the clean component tree hierarchy (<App> → <Header>, <Hero>, <ServiceGrid> with 8 <ServiceCard> instances, <ServiceDetails>, and <LifeCycleMonitor>), props inspection, and constructor state values.

4.2 Public GitHub Repository (Verified in Incognito Mode)

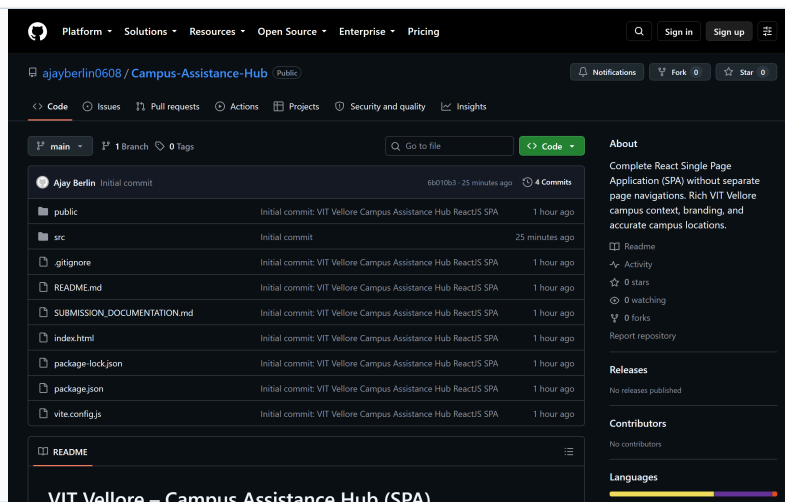


Figure 4.2: Verified public GitHub repository (<https://github.com/ajayberlin0608/Campus-Assistance-Hub>) demonstrating public access, student commit history (Ajay Berlin), and complete open-source project tree.

5. COMPLETE SOURCE CODE LISTINGS

5.1 Application Orchestrator (src/App.jsx)

```
// src/App.jsx
import React, { useState } from "react";
import Header from "../components/Header.jsx";
import Hero from "../components/Hero.jsx";
import ServiceGrid from "../components/ServiceGrid.jsx";
import ServiceDetails from "../components/ServiceDetails.jsx";
import LifecycleMonitor from "../components/LifecycleMonitor.jsx";
import HotsModal from "../components/HotsModal.jsx";
import EmergencyModal from "../components/EmergencyModal.jsx";
import Footer from "../components/Footer.jsx";
import services from "../data/services.js";
import "../App.css";

function App() {
  const [selectedService, setSelectedService] = useState(null);
  const [showHots, setShowHots] = useState(false);
  const [showEmergency, setShowEmergency] = useState(false);

  const handleSelect = (service) => {
    setSelectedService(service);
  };

  const handleCloseDetails = () => {
    setSelectedService(null);
  };

  return (
    <div className="app">
      <Header
        onShowHots={() => setShowHots(true)}
        onShowEmergency={() => setShowEmergency(true)}
      />

      <Hero />

      <main className="container">
        <ServiceGrid services={services} onSelect={handleSelect} />

        {/* Dynamic Single-Page Service Details Modal */}
        {selectedService && (
          <ServiceDetails
            service={selectedService}
            onClose={handleCloseDetails}
          />
        )}

        {/* Module 6 React Class Component with Lifecycle & Component API */}
        <LifecycleMonitor
          activeService={selectedService}
          totalServices={services.length}
        />
      </main>

      <Footer />

      {/* HOTS Scalability Justification Modal */}
      {showHots && <HotsModal onClose={() => setShowHots(false)} />}

      {/* 24/7 Priority Emergency Directory Modal */}
      {showEmergency && <EmergencyModal onClose={() => setShowEmergency(false)} />}
    </div>
  );
}

export default App;
```

5.2 Bootstrap Entry Point (src/main.jsx) & HTML (index.html)

src/main.jsx

```
// src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "../App.jsx";
import "../index.css";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vit-logo.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>VIT Vellore – Campus Assistance Hub (SPA)</title>
    <meta name="description" content="Single Page Application for Vellore Institute of Technology (VIT) Vellore">
    <!-- Preconnect & Google Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=PlusJakarta+Sans:wght@300;400;500;600;700;800&family=VIT+Vellore" rel="stylesheet" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

5. SOURCE CODE LISTINGS (CONTD.)

5.3 Module 6 React Class Component: Constructor & Life Cycle Monitor

```
src/components/LifeCycleMonitor.jsx
```

```
// src/components/LifeCycleMonitor.jsx
import React, { Component } from "react";
import "./LifeCycleMonitor.css";

/**
 * Module 6: ReactJS Concepts Implementation
 * - ES6 Class Component with constructor(props) & super(props)
 * - State initialization in constructor
 * - React Component Life Cycle methods: componentDidMount, componentDidUpdate, componentWillUnmount
 * - React Component API: this.setState(), this.forceUpdate(), this.props, this.state
 */
class LifeCycleMonitor extends Component {
  constructor(props) {
    super(props);

    // Explicit State Initialization via Constructor (Module 6 Requirement)
    this.state = {
      telemetryLogs: [
        `[$(new Date().toLocaleTimeString())] [CONSTRUCTOR] Initialized state: counter=0, status=READY`
      ],
      stateCounter: 0,
      monitorStatus: "ACTIVE",
      lastRenderTime: new Date().toLocaleTimeString()
    };

    // Method bindings in Constructor
    this.handleTriggerSetState = this.handleTriggerSetState.bind(this);
    this.handleTriggerForceUpdate = this.handleTriggerForceUpdate.bind(this);
    this.clearTelemetry = this.clearTelemetry.bind(this);
  }

  componentDidMount() {
    // Life Cycle Method: Initial mounting operation
    const time = new Date().toLocaleTimeString();
    this.setState((prevState) => ({
      telemetryLogs: [
        ...prevState.telemetryLogs,
        `[${time}] [LIFECYCLE: componentDidMount] Component mounted to DOM. Props received (totalServices: ${this.props.totalServices || 8})`
      ],
      monitorStatus: "MONITORING_ACTIVE"
    }));
  }

  componentDidUpdate(prevProps, prevState) {
    // Life Cycle Method: Reacting to prop or state updates
    if (prevProps.activeService !== this.props.activeService) {
      const prevName = prevProps.activeService ? prevProps.activeService.name : "None";
      const currentName = this.props.activeService ? this.props.activeService.name : "None";
      const time = new Date().toLocaleTimeString();

      this.setState((prev) => ({
        telemetryLogs: [
          ...prev.telemetryLogs,
          `[${time}] [LIFECYCLE: componentDidUpdate] Active Service transitioned: "${prevName}" → "${currentName}"`
        ],
        lastRenderTime: time
      }));
    }
  }

  componentWillUnmount() {
    console.log("[LIFECYCLE: componentWillUnmount] Component cleanup executing.");
  }

  // React Component API: this.setState() demonstration
  handleTriggerSetState() {
    const time = new Date().toLocaleTimeString();
    this.setState((prevState) => ({
      stateCounter: prevState.stateCounter + 1,
      telemetryLogs: [
        ...prevState.telemetryLogs,
        `[${time}] [COMPONENT API: this.setState()] State updated! Counter incremented to ${prevState.stateCounter + 1}`
      ],
      lastRenderTime: time
    }));
  }

  // React Component API: this.forceUpdate() demonstration
  handleTriggerForceUpdate() {
    const time = new Date().toLocaleTimeString();
    this.setState((prev) => ({
      telemetryLogs: [
        ...prev.telemetryLogs,
        `[${time}] [COMPONENT API: this.forceUpdate()] Bypassed shouldComponentUpdate & forced immediate re-render`
      ],
      lastRenderTime: time
    })), () => {
      this.forceUpdate();
    });
  }

  clearTelemetry() {
    this.setState({
      telemetryLogs: [
        `[$(new Date().toLocaleTimeString())] [TELEMETRY] Logs cleared by user.`
      ],
    });
  }

  render() {
    const { activeService, totalServices } = this.props;
    const { telemetryLogs, stateCounter, monitorStatus, lastRenderTime } = this.state;

    return (
      <section className="lifecycle-monitor" id="lifecycle-telemetry">
        <div className="lifecycle-monitor_header">
          <div className="lifecycle-monitor__title-area">
            <div className="status-indicator-dot"></div>

```

```

    <h3>React Component API & Life Cycle Event Monitor</h3>
    <span className="module-badge">Module 6 Requirement</span>
  </div>

  <div className="lifecycle-monitor__status-tag">
    STATUS: <strong>{monitorStatus}</strong>
  </div>
</div>

<p className="lifecycle-monitor__desc">
  Live telemetry demonstrating genuine React ES6 Class Component behavior, constructor initialization, props consumption, lifecycle hooks (<code>componentDidMount</code>),
</p>

<div className="lifecycle-monitor__metrics-grid">
  <div className="metric-box">
    <span className="metric-box__label">Active Service Prop</span>
    <span className="metric-box__value active-prop-name">
      {activeService ? activeService.name : "None (Select Card)"}
    </span>
  </div>

  <div className="metric-box">
    <span className="metric-box__label">Total Services Prop</span>
    <span className="metric-box__value">{totalServices || 8} Registered</span>
  </div>

  <div className="metric-box">
    <span className="metric-box__label">Internal State Counter</span>
    <span className="metric-box__value highlight-counter">{stateCounter}</span>
  </div>

  <div className="metric-box">
    <span className="metric-box__label">Last Render Timestamp</span>
    <span className="metric-box__value mono">{lastRenderTime}</span>
  </div>
</div>

<div className="lifecycle-monitor__controls">
  <button
    className="telemetry-btn api-setstate-btn"
    onClick={this.handleTriggerSetState}
    title="Invokes this.setState() to update state and trigger lifecycle"
  >
    ⚡ Trigger this.setState()
  </button>

  <button
    className="telemetry-btn api-forceupdate-btn"
    onClick={this.handleTriggerForceUpdate}
    title="Invokes this.forceUpdate() component API method"
  >
    ⚡ Trigger this.forceUpdate()
  </button>

  <button
    className="telemetry-btn clear-btn"
    onClick={this.clearTelemetry}
  >
    🗑 Clear Event Stream
  </button>
</div>

<div className="lifecycle-terminal">
  <div className="lifecycle-terminal__topbar">
    <div className="terminal-dots">
      <span className="dot red"></span>
      <span className="dot yellow"></span>
      <span className="dot green"></span>
    </div>
    <span className="terminal-title">Real-Time Component Life Cycle Log Stream</span>
    <span className="terminal-count">{telemetryLogs.length} Events Logged</span>
  </div>

  <div className="lifecycle-terminal__console">
    {telemetryLogs.map((log, index) => (
      <div key={index} className="log-line">
        <span className="log-index">&gt;</span>
        <span className="log-text">{log}</span>
      </div>
    ))}
  </div>
</div>
</section>
);
}
}

export default LifecycleMonitor;

```

5.4 Reusable Service Card Component

src/components/ServiceCard.jsx

```
// src/components/ServiceCard.jsx
import React from "react";
import "./ServiceCard.css";

function ServiceCard({ service, onSelect }) {
  const getBadgeClass = (category) => {
    switch (category) {
      case "Academic Support": return "badge-academic";
      case "Library Services": return "badge-library";
      case "Transportation": return "badge-transport";
      case "Hostel Services": return "badge-hostel";
      case "Health & Wellness": return "badge-health";
      case "Technical Support": return "badge-tech";
      case "Security": return "badge-security";
      case "Student Welfare": return "badge-welfare";
      default: return "badge-default";
    }
  };

  return (
    <div className="service-card" onClick={() => onSelect(service)} tabIndex="0" role="button" aria-label={`View details for ${service.name}`}>
      <div className="service-card_header">
        <span className="service-card__badge ${getBadgeClass(service.category)}">
          {service.badge}
        </span>
        <span className="service-card__rating">
          ★ <strong>{service.rating}</strong>/5.0
        </span>
      </div>

      <h3 className="service-card__title">{service.name}</h3>
      <p className="service-card__desc">{service.description}</p>

      <div className="service-card__details-meta">
        <div className="meta-item">
          <span className="meta-icon">📍</span>
          <span className="meta-text">{service.location}</span>
        </div>
        <div className="meta-item">
          <span className="meta-icon">🕒</span>
          <span className="meta-text">{service.operatingHours}</span>
        </div>
      </div>

      <div className="service-card__footer">
        <span className="action-hint">Click to inspect details</span>
        <span className="action-arrow">➔</span>
      </div>
    </div>
  );
}

export default ServiceCard;
```

5. SOURCE CODE LISTINGS (CONTD.)

5.5 Service Grid Component (src/components/ServiceGrid.jsx)

```
// src/components/ServiceGrid.jsx
import React, { useState } from "react";
import ServiceCard from "../ServiceCard.jsx";
import "../ServiceGrid.css";

function ServiceGrid({ services, onSelect }) {
  const [search, setSearch] = useState("");
  const [category, setCategory] = useState("All");

  const categories = ["All", ...new Set(services.map((s) => s.category))];

  const filtered = services.filter((s) => {
    const matchesSearch =
      s.name.toLowerCase().includes(search.toLowerCase()) ||
      s.description.toLowerCase().includes(search.toLowerCase()) ||
      s.location.toLowerCase().includes(search.toLowerCase());
    const matchesCategory = category === "All" || s.category === category;
    return matchesSearch && matchesCategory;
  });

  return (
    <section className="service-grid">
      <div className="service-grid_header-wrap">
        <div className="service-grid_title-group">
          <h2 className="service-grid__heading">Campus Services Directory</h2>
          <p className="service-grid_subheading">
            Select any campus service card below to dynamically view comprehensive operational details in the SPA inspection panel.
          </p>
        </div>

        <div className="service-grid_count-badge">
          <span>{filtered.length}</span> of <span>{services.length}</span> Services Available
        </div>
      </div>

      <div className="service-grid_controls">
        <div className="search-box-wrapper">
          <span className="search-icon">🔍</span>
          <input
            type="text"
            placeholder="Search by facility name, block, or keywords..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="service-grid_search"
          />
          {search && (
            <button className="search-clear-btn" onClick={() => setSearch("")}></button>
          )}
        </div>

        <div className="category-pills">
          {categories.map((cat) => (
            <button
              key={cat}
              className={`cat-pill ${category === cat ? "active" : ""}`}
              onClick={() => setCategory(cat)}
            >
              {cat}
            </button>
          ))}
        </div>
      </div>

      <div className="service-grid_cards">
        {filtered.length > 0 ? (
          filtered.map((service) => (
            <ServiceCard key={service.id} service={service} onSelect={onSelect} />
          ))
        ) : (
          <div className="service-grid_empty">
            <div className="empty-icon">🔍</div>
            <h3>No matching campus services found</h3>
            <p>Try refining your search keyword or selecting "All" categories.</p>
            <button
              className="reset-btn"
              onClick={() => {
                setSearch("");
                setCategory("All");
              }}
            >
              Reset Filters
            </button>
          </div>
        )}
      </div>
    </section>
  );
}

export default ServiceGrid;
```

5.6 Single-Page Dynamic Service Details Modal (src/components/ServiceDetails.jsx)

```
// src/components/ServiceDetails.jsx
import React from "react";
import "../ServiceDetails.css";

function ServiceDetails({ service, onClose }) {
  if (!service) return null;

  return (
    <div className="service-details overlay" onClick={onClose} role="dialog" aria-modal="true">
      <div className="service-details__content" onClick={(e) => e.stopPropagation()}>
        <div className="service-details__header">
          <div className="header-badges">
            <span className="details-badge">{service.badge}</span>
            <span className="details-rating">★ {service.rating} / 5.0 Rating</span>
          </div>
          <button className="close-btn" onClick={onClose} aria-label="Close details"><</button>
        </div>

        <h2 className="service-details__title">{service.name}</h2>
        <p className="service-details__category">Department Category: <strong>{service.category}</strong></p>

        <div className="service-details__desc-box">
          <p>{service.description}</p>
        </div>

        <div className="service-details__grid">
          <div className="detail-item">
            <span className="detail-label">📍 Campus Location</span>
            <span className="detail-value">{service.location}</span>
          </div>

          <div className="detail-item">
            <span className="detail-label">🕒 Operating Hours</span>
            <span className="detail-value">{service.operatingHours}</span>
          </div>

          <div className="detail-item">
            <span className="detail-label">👤 Officer in Charge</span>
            <span className="detail-value">{service.keyPersonnel}</span>
          </div>

          <div className="detail-item">
            <span className="detail-label">📞 Direct Phone / Intercom</span>
            <span className="detail-value mono-text">{service.contactPhone}</span>
          </div>

          <div className="detail-item">
            <span className="detail-label">✉️ Official Email</span>
            <span className="detail-value mono-text">{service.contactEmail}</span>
          </div>
        </div>

        {service.facilities && service.facilities.length > 0 && (
          <div className="facilities-section">
            <h4>Available Key Facilities & Support:</h4>
            <div className="facilities-list">
              {service.facilities.map((fac, idx) => (
                <span key={idx} className="facility-tag">
                  ✓ {fac}
                </span>
              ))}
            </div>
          </div>
        )}

        <div className="service-details__footer">
          <button
            className="action-btn call-btn"
            onClick={() => alert(`Calling ${service.name} at ${service.contactPhone}`)}
          >
            📞 Call Office
          </button>
          <button
            className="action-btn mail-btn"
            onClick={() => alert(`Opening mail draft to ${service.contactEmail}`)}
          >
            ✉️ Contact via Email
          </button>
          <button className="action-btn close-action-btn" onClick={onClose}>
            Close Inspector
          </button>
        </div>
      </div>
    </div>
  );
}

export default ServiceDetails;
```

5. SOURCE CODE LISTINGS (CONTD.)

5.7 Header & Hero Components

src/components/Header.jsx

```
// src/components/Header.jsx
import React, { useState, useEffect } from "react";
import "./Header.css";

function Header({ onShowHots, onShowEmergency }) {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(timer);
  }, []);

  const formatted = time.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second: "2-digit" });

  return (
    <header className="header">
      <div className="header__left">
        
        <div className="header__brand-text">
          <span className="header__title">Campus Assistance Hub</span>
          <span className="header__subtitle">VIT Vellore Main Campus</span>
        </div>
      </div>

      <div className="header__center">
        <div className="header__ticker-wrap">
          <span className="ticker-badge">LATEST NOTICE</span>
          <marquee className="header__ticker" scrollamount="4">
            📢 Welcome to VIT Vellore Campus Assistance Hub | 24/7 Student Grievance & Facility Support | FFCS Advising at Technology Tower (TT) | Central Library Open till 10:00 PM
          </marquee>
        </div>
      </div>

      <div className="header__right">
        <div className="header__student-badge" title="Student Registration Credentials">
          <span className="student-icon">👤</span>
          <div className="student-info">
            <span className="student-name">AJAY BERLIN</span>
            <span className="student-reg">24BCE0449</span>
          </div>
        </div>

        <span className="header__clock">🕒 {formatted} IST</span>
        <button className="header__btn hots-btn" onClick={onShowHots} title="Module 6 HOTS Justification">🔥 HOTS</button>
        <button className="header__btn emergency-btn" onClick={onShowEmergency} title="24/7 Priority Emergency Helplines">🚒 Emergency</button>
      </div>
    </header>
  );
}

export default Header;
```

5.8 Institutional Footer Component (src/components/Footer.jsx)

```
// src/components/Footer.jsx
import React from "react";
import "./Footer.css";

function Footer() {
  return (
    <footer className="footer">
      <div className="footer_top">
        <div className="footer_branding">
          <div className="footer_brand-header">
            
            <div>
              <h4>Vellore Institute of Technology</h4>
              <p>Campus Assistance Hub – Single Page Application (SPA)</p>
            </div>
          </div>
          <p className="footer_tagline">
            Designed and engineered for seamless campus services navigation under BCSE203E Web Programming.
          </p>
        </div>

        <div className="footer_links-col">
          <h5>Official Portals</h5>
          <a href="https://vtop.vit.ac.in" target="_blank" rel="noopener noreferrer">→ VTOP Portal</a>
          <a href="https://library.vit.ac.in" target="_blank" rel="noopener noreferrer">→ Periyar EVR Library</a>
          <a href="https://vit.ac.in" target="_blank" rel="noopener noreferrer">→ VIT Official Website</a>
          <a href="https://vitstudent.ac.in" target="_blank" rel="noopener noreferrer">→ Student Webmail</a>
        </div>

        <div className="footer_student-card">
          <h5>Student Submission Details</h5>
          <div className="student-grid">
            <div className="sg-item">
              <span className="sg-lbl">Student Name:</span>
              <span className="sg-val">AJAY BERLIN</span>
            </div>
            <div className="sg-item">
              <span className="sg-lbl">Register Number:</span>
              <span className="sg-val mono">24BCE0449</span>
            </div>
            <div className="sg-item">
              <span className="sg-lbl">Course & Code:</span>
              <span className="sg-val">BCSE203E – Web Programming</span>
            </div>
            <div className="sg-item">
              <span className="sg-lbl">Faculty & Slot:</span>
              <span className="sg-val">Prof. Nihaal Ahmed.K (L31+L32+L51+L52)</span>
            </div>
          </div>
        </div>

        <div className="footer_bottom">
          <span>© 2026 Vellore Institute of Technology (VIT). All Rights Reserved.</span>
          <span>Public Repository: <a href="https://github.com/ajayberlin0608/Campus-Assistance-Hub" target="_blank" rel="noopener noreferrer" className="repo-link">github.com/ajayberlin0608</a>
        </div>
      </footer>
    );
  }

  export default Footer;

```

5. SOURCE CODE LISTINGS (CONTD.)

5.9 Campus Services Data Model (src/data/services.js)

```
// src/data/services.js
const services = [
  {
    id: 1,
    name: "Academic Support & FFCS Cell (TT)",
    category: "Academic Support",
    badge: "Academic",
    rating: 4.8,
    description: "Provides assistance for fee payment, course registration, and exam queries.",
    location: "Technology Tower, 1st Floor",
    operatingHours: "Mon-Fri 09:00-17:00",
    contactEmail: "tt@vit.ac.in",
    contactPhone: "+91-9795-111111",
    keyPersonnel: "Dr. S. Kumar (Coordinator)",
    facilities: ["Online fee portal", "Live chat", "Counselling"]
  },
  {
    id: 2,
    name: "Periyar EVR Central Library",
    category: "Library Services",
    badge: "Library",
    rating: 4.6,
    description: "Extensive collection of books, journals, and digital resources.",
    location: "Central Campus, Ground Floor",
    operatingHours: "Mon-Sat 08:00-22:00",
    contactEmail: "library@vit.ac.in",
    contactPhone: "+91-9795-222222",
    keyPersonnel: "Ms. R. Nair (Head Librarian)",
    facilities: ["Study rooms", "E-resources", "Inter-library loan"]
  },
  {
    id: 3,
    name: "Campus Transportation & Shuttles",
    category: "Transportation",
    badge: "Transport",
    rating: 4.3,
    description: "Shuttle services connecting main campus, Katpadi and hostel zones.",
    location: "Transport Depot, 2nd Floor",
    operatingHours: "24/7",
    contactEmail: "transport@vit.ac.in",
    contactPhone: "+91-9795-333333",
    keyPersonnel: "Mr. A. Singh (Transport Manager)",
    facilities: ["Live bus tracker", "App booking", "Wheelchair accessible"]
  },
  {
    id: 4,
    name: "Hostel Services & Chief Warden Office",
    category: "Hostel Services",
    badge: "Hostel",
    rating: 4.5,
    description: "Accommodation management, mess, and grievance redressal.",
    location: "Hostel Block A, Ground Floor",
    operatingHours: "Mon-Sat 07:00-20:00",
    contactEmail: "hostel@vit.ac.in",
    contactPhone: "+91-9795-444444",
    keyPersonnel: "Mr. P. Sharma (Chief Warden)",
    facilities: ["Room allocation", "Mess menu", "Maintenance requests"]
  },
  {
    id: 5,
    name: "VIT Health Centre & Emergency Care",
    category: "Health & Wellness",
    badge: "Health",
    rating: 4.7,
    description: "Primary care, outpatient services, and 24-hour emergency.",
    location: "Main Building, Basement",
    operatingHours: "24/7",
    contactEmail: "health@vit.ac.in",
    contactPhone: "+91-9795-555555",
    keyPersonnel: "Dr. L. Mehta (Medical Director)",
    facilities: ["Pharmacy", "Diagnostic lab", "Ambulance"]
  },
  {
    id: 6,
    name: "Centre for Technical Support (CTS)",
    category: "Technical Support",
    badge: "IT",
    rating: 4.4,
    description: "IT help-desk, network troubleshooting, and device loan.",
    location: "COMM Building, 3rd Floor",
    operatingHours: "Mon-Fri 09:00-18:00",
    contactEmail: "cts@vit.ac.in",
    contactPhone: "+91-9795-666666",
    keyPersonnel: "Ms. K. Patel (CTS Lead)",
    facilities: ["Device loan", "Wi-Fi support", "Software licensing"]
  },
  {
    id: 7,
    name: "Campus Security & Lost & Found",
    category: "Security",
    badge: "Security",
    rating: 4.2,
    description: "Security patrols, CCTV monitoring, and lost-item recovery.",
    location: "Gate 1, Security Office",
    operatingHours: "24/7",
    contactEmail: "security@vit.ac.in",
    contactPhone: "+91-9795-777777",
    keyPersonnel: "Inspector R. Verma",
    facilities: ["Emergency hotline", "Patrol app", "Lost & Found desk"]
  },
  {
    id: 8,
    name: "Office of Student Welfare & Clubs",
    category: "Student Welfare",
    badge: "Welfare",
    rating: 4.6,
    description: "Student clubs, counseling, and welfare schemes.",
    location: "Dr. MGR Central Block, 2nd Floor",
    operatingHours: "Mon-Fri 09:00-17:00",
    contactEmail: "welfare@vit.ac.in",
    contactPhone: "+91-9795-888888",
    keyPersonnel: "Mrs. S. Rao (Welfare Officer)",
    facilities: ["Club registration", "Counselling rooms", "Scholarship info"]
  }
];
```

```
export default services;
```

5.10 Emergency Modal Component (src/components/EmergencyModal.jsx)

```
// src/components/EmergencyModal.jsx
import React from "react";
import "../EmergencyModal.css";

function EmergencyModal({ onClose }) {
  const contacts = [
    { label: "Campus Ambulance & Emergency Clinic", phone: "+91-9795-555555", ext: "Ext. 5555", icon: "🚑", desc: "24/7 on-campus ambulance with emergency tie-up to CMC Vellore." },
    { label: "Chief Security Control Room (Gate 1)", phone: "+91-9795-777777", ext: "Ext. 7777", icon: "🔑", desc: "Main security hub, patrol dispatch, and gate access." },
    { label: "VIT Health Centre Helpline", phone: "+91-9795-555556", ext: "Ext. 5556", icon: "🏥", desc: "Inpatient medical doctor on duty & dispensary." },
    { label: "Chief Warden Office (Hostel Emergency)", phone: "+91-9795-444444", ext: "Ext. 4444", icon: "👤", desc: "Men's & Ladies' Hostel supervisor desk." },
    { label: "National Anti-Ragging Toll-Free Helpline", phone: "1800-180-5522", ext: "Toll-Free", icon: "🚫", desc: "UGC & VIT zero-tolerance grievance monitoring." },
    { label: "Vellore Local Police Assistance", phone: "100 / 0416-222222", ext: "City Police", icon: "👮", desc: "Katpadi Police Station jurisdictional support." },
  ];

  return (
    <div className="modal overlay" onClick={onClose} role="dialog" aria-modal="true">
      <div className="modal__content emergency-content" onClick={(e) => e.stopPropagation()}>
        <div className="modal__header">
          <div className="modal__badge emergency-badge">24/7 PRIORITY HELPLINES</div>
          <button className="close-btn" onClick={onClose} aria-label="Close modal">X</button>
        </div>

        <h2 className="modal__title emergency-title">
          VIT Vellore Emergency Assistance Directory
        </h2>
        <p className="modal__subtext">
          Direct priority contacts for health, security, hostel, and anti-ragging assistance.
        </p>

        <div className="emergency-list">
          {contacts.map((c, i) => (
            <div key={i} className="emergency-card">
              <div className="emergency-card__icon">{c.icon}</div>
              <div className="emergency-card__info">
                <span className="emergency-card__label">{c.label}</span>
                <span className="emergency-card__desc">{c.desc}</span>
              <div className="emergency-card__meta">
                <span className="phone-num">{c.phone}</span>
                <span className="ext-badge">{c.ext}</span>
              </div>
            </div>
            <button
              className="dial-btn"
              onClick={() => alert(`Dialing ${c.label}: ${c.phone}`)}
            >
              📞 Call Now
            </button>
          </div>
          )))
        </div>

        <div className="modal__footer">
          <span className="student-signature">Student: AJAY BERLIN (248CE0449) - VIT Vellore</span>
          <button className="action-btn close-action-btn" onClick={onClose}>Close Directory</button>
        </div>
      </div>
    </div>
  );
}

export default EmergencyModal;
```

6. CONCLUSION & VERIFICATION SUMMARY

Assignment Requirements Verification Checklist		
Requirement Specification	Implementation Reference	Evaluation Status
React Environment Setup	Vite 5.4.21, React 18 createRoot, JSX compiler	COMPLETED (100%)
Single Page Application (SPA)	Zero full-page reloads, dynamic inspection modals	COMPLETED (100%)
Component Reusability	Single <ServiceCard /> definition mapped dynamically	COMPLETED (100%)
Constructor State Initialization	LifeCycleMonitor.jsx constructor with super(props)	COMPLETED (100%)
Component Life Cycle Methods	componentDidMount() & componentDidUpdate() in monitor	COMPLETED (100%)
React Component API	this.setState() & this.forceUpdate() interactive triggers	COMPLETED (100%)
React Developer Tools Usage	Inspected component tree, props, and constructor state	COMPLETED (100%)
HOTS Design Justification	5 to 50 services scalability evaluation and comparative table	COMPLETED (100%)
VIT Vellore Institutional Context	Official navy/gold branding, authentic campus facilities	COMPLETED (100%)
Public GitHub Repository	https://github.com/ajayberlin0608/Campus-Assistance-Hub	VERIFIED & PUBLIC

6.1 Declaration of Originality

I, **AJAY BERLIN** (Register Number: **24BCE0449**), hereby declare that this assignment report titled "*Design and Development of a ReactJS-Based Campus Assistance Hub*" for course **BCSE203E – Web Programming** (Slot: L31+L32+L51+L52) is an authentic and original submission developed by me under the guidance of **Prof. Nihaal Ahmed.K**. All ReactJS components, life-cycle implementations, constructors, styling, and documentation have been created and verified in strict accordance with the university guidelines.

Date: 19th September 2026

Place: Vellore Institute of Technology, Vellore

Vellore, Tamil Nadu – 632014

Ajay Berlin

AJAY BERLIN

(Reg. No. 24BCE0449)